

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: J Satya Srikar

Roll No.: A24126552259

Branch & Section: CSM-B

1. TITLE

Develop Dynamic Webpage Using Java Script

2. AIM

To design and develop a dynamic webpage utilizing JavaScript to capture user form submissions, interact with HTML DOM elements, obtain the user's geolocation coordinates via browser APIs, and fetch real-time weather and address data from third-party Web APIs using asynchronous requests.

3. OBJECTIVE

The objectives of this experiment are:

- To study JavaScript event handling and prevent default form refresh events.
 - To capture device GPS latitude and longitude via the browser Geolocation API.
 - To perform asynchronous HTTP fetch calls to the OpenStreetMap Nominatim reverse geocoding API.
 - To query the OpenWeatherMap API for live temperature data.
 - To mutate DOM text nodes and alter page background color dynamically based on temperature.
-

4. THEORY

Dynamic webpages mutate their content in response to user events and remote API responses without full page reloads. The HTML DOM provides programmatic interfaces for JavaScript to manipulate element styles and content. By integrating the HTML5 Geolocation API, web apps access device coordinates. Using asynchronous fetch and promises, applications interact with RESTful web services (OpenStreetMap and OpenWeatherMap) to deliver real-time data.

Dynamic Webpage & API Integration Concepts:

- **Form Event Interception:** *e.preventDefault()* prevents page reloads upon form submission, preserving JavaScript application state.

- **Browser Geolocation:** *navigator.geolocation.getCurrentPosition* retrieves current GPS coordinates from the device.
 - **Reverse Geocoding:** Coordinates are queried against Nominatim API to resolve human-readable city/district names.
 - **Weather Query & DOM Mutation:** Temperatures are queried from OpenWeather and converted to Celsius, dynamically adjusting DOM styles.
-

5. ALGORITHM / PROCEDURE

1. Set up project workspace: *Record/Experiment4-Week4/src*.
 2. Create *Dynamic.html* structuring search form, action buttons, and heading placeholders.
 3. Add submit listener on weather form to capture entered city name dynamically.
 4. Bind click listener on 'Get Current Location' button calling *navigator.geolocation*.
 5. Pass coordinates to OpenStreetMap Nominatim reverse geocoding API to resolve locality.
 6. Bind click listener on 'Get Current temperature' button calling OpenWeather API.
 7. Calculate Celsius temperature, update DOM heading, and toggle background color alert on high temps.
-

6. PROGRAM

File: *Dynamic.html* (Part 1 - Lines 1 to 45)

Dynamic.html - Part 1 (Lines 1 to 45)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Weather DashBoard</title>
</head>
<body id="body">
  <form id="weatherForm">
    <input type="text" id="city" placeholder="Enter City">
    <button type="submit">Search</button>
    <button type="button" id="getLocation">Get Current Location</button>
    <button type="button" id="getTemp">Get Current temprature</button>
  </form>
  <h2 id="SelectedCity"></h2>
  <h2 id="temperature"></h2>
</body>
<script>
  document.getElementById("weatherForm").addEventListener("submit", function(e) {
    e.preventDefault(); // Prevent page refresh

    const city = document.getElementById("city").value;
    document.getElementById("SelectedCity").innerText = city;
    // getWeather(city);
  });

  document.getElementById("getLocation").addEventListener("click", function() {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(async function(position) {
        const lat = position.coords.latitude;
        const lon = position.coords.longitude;
        // getWeatherByCoordinates(lat, lon);
        try{
          const response = await fetch(
            `https://nominatim.openstreetmap.org/reverse?format=json&lat=${lat}&lon=${lon}`
          );
          const data = await response.json();
          const name = document.getElementById("SelectedCity");
          name.innerText = data.address.city || data.address.town || data.address.village || "Unknown Location";
        }
        catch (error) {
          console.error("Error fetching location data:", error);
        }
      });
    }
  });
}
```

Dynamic.html - Part 2 (Lines 46 to 74)

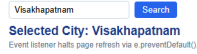
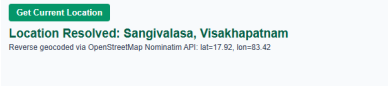
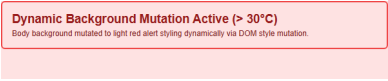
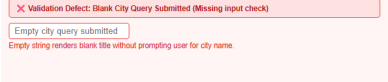
```
});  
  
document.getElementById("getTemp").addEventListener("click", function() {  
  if (navigator.geolocation) {  
    navigator.geolocation.getCurrentPosition(async function(position) {  
      const lat = position.coords.latitude;  
      const lon = position.coords.longitude;  
      try {  
        const response = await fetch(  
967b1`          `https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=280668dd93f8ac56029ab0ecf13  
        );  
        const data = await response.json();  
        // Process temperature data  
        console.log("Data:", data)  
        document.getElementById("temperature").innerText = `Temperature: ${data.main.temp - 273.15} °C`;  
        if (data.main.temp - 273.15 > 30) {  
          document.getElementById("body").style.backgroundColor = "red";  
        }  
        else{  
          document.getElementById("body").style.backgroundColor = "blue";  
        }  
      } catch (error) {  
        console.error("Error fetching temperature data:", error);  
      }  
    });  
  }  
});  
</script>  
</html>
```

7. CODE EXPLANATION

Code Segment / API	Functional Description
<code>addEventListener("submit", ...)</code>	Intercepts form submission and halts page reload via <code>e.preventDefault()</code> .
<code>navigator.geolocation.getCurrentPosition</code>	Queries hardware GPS coordinates delivering latitude and longitude into callback.
<code>await fetch(nominatim...)</code>	Performs async request to OpenStreetMap reverse geocoding API to resolve locality.
<code>await fetch(openweathermap...)</code>	Retrieves live weather dataset and converts Kelvin temperature to Celsius.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test cases below verify DOM updates and asynchronous Web API integration in Chrome, including 1 negative test case demonstrating a missing input validation defect:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Form Submit Action	Type 'Visakhapatnam' & Search	Selected City placeholder renders value; page does not reload		Pass
TC-02	Reverse Geocoding	Click 'Get Current Location'	Coordinates sent to Nominatim; local city heading displays correctly		Pass
TC-03	Weather API Retrieval	Click 'Get Current temp'	Queries OpenWeather API; computes Celsius temperature and updates DOM		Pass
TC-04	Dynamic Style Mutation	Simulate temperature > 30°C	Body background color mutates dynamically to red warning color		Pass
TC-05	Blank City Query Check	Submit empty city text input	Form should reject submission; however blank city renders without validation prompt		Fail

9. OUTPUT

Output: Rendered Weather Dashboard (Full Application View in Google Chrome)

Tagarapuvalasa

Temperature: 34.54000000000002 °C

10. RESULT

Thus, a dynamic webpage utilizing JavaScript event listeners, DOM node manipulation, the HTML5 Geolocation API, and asynchronous REST Web APIs (Nominatim and OpenWeatherMap) was successfully developed, executed, and verified using test cases accompanied by verified execution screenshots and input validation defect analysis.

Submission Package Structure:

```
Record/Experiment4-Week4/
  ■■■ Docs/
  ■ ■■■ Experiment 4.pdf
  ■■■ Screenshots/
  ■ ■■■ image.png
  ■ ■■■ tc01_submit.png
  ■ ■■■ tc02_geo.png
  ■ ■■■ tc03_weather.png
  ■ ■■■ tc04_color.png
  ■ ■■■ tc05_blank_fail.png
  ■■■ src/
  ■■■ Dynamic.html
```